Politechnika Warszawska
Wydział Matematyki i Nauk Informacyjnych

Projekt 1: Klasyfikacja i regresja z użyciem perceptronu

Przedmiot: Sieci Neuronowe

Autorzy:
Jan Najowski
Stefan Stawicki

Prowadzący:
[Tytuł / Stopień, Imię i Nazwisko]

Warszawa, 18 października 2026

Spis treści

1 Wstęp i cel projektu **1**

2 Implementacja wielowarstwowego perceptronu (MLP) **1**

3 Eksperymenty — Zagadnienie regresji **2**
3.1 Zbiory danych i metodyka badawcza 2
3.2 Wpływ funkcji aktywacji w warstwach ukrytych 2
3.3 Wpływ architektury sieci: głębokość i szerokość 2
3.4 Wpływ miary błędu (funkcji straty) 2
3.5 Wpływ rozmiaru zbioru danych (opcjonalnie) 2
3.6 Podsumowanie i wnioski z części regresyjnej 2

4 Eksperymenty — Zagadnienie klasyfikacji **2**
4.1 Zbiory danych i metodyka badawcza 2
4.2 Wpływ funkcji aktywacji w warstwach ukrytych 2
4.3 Wpływ architektury sieci: głębokość i szerokość 2
4.4 Wpływ miary błędu (funkcji straty) 2
4.5 Podsumowanie i wnioski z części klasyfikacyjnej 2

5 Podsumowanie ogólne i wnioski końcowe **2**

1 Wstęp i cel projektu

Celem niniejszego projektu jest samodzielna implementacja wielowarstwowego perceptronu (*Multilayer Perceptron*, MLP) od podstaw z wykorzystaniem wyłącznie biblioteki `NumPy`. W ramach implementacji, przygotowano algorytm wstecznej propagacji błędu oraz stochastycznego spadku gradientu (*Stochastic Gradient Descent*). Przygotowany model posłużył do rozwiązania zagadnień aproksymacji nieliniowej dla funkcji $\mathbb{R} \to \mathbb{R}$ (zadanie regresji) oraz przydzielania klas punktom w przestrzeni $\mathbb{R}^2$ (zadanie klasyfikacji binarnej i wieloklasowej). W ramach pracy zbadano kilka aspektów wpływających na działanie sieci, takich jak dobór funkcji aktywacji (Sigmoid, Tanh, ReLU), głębokości i szerokości architektury (od 0 do 4 warstw ukrytych) oraz zastosowanych miar błędu na wyjściu modelu (MSE, BCE, CCE).

2 Implementacja wielowarstwowego perceptronu (MLP)

Kod źródłowy został podzielony na następujące moduły:

- **layers** — definiuje klasę bazową warstwy oraz warstwę w pełni połączoną (`Dense`), odpowiedzialną za transformację liniową, przechowywanie wag i biasów, a także inicjalizację wag przy pomocy zdefiniowanych rozkładów [1].
- **activations** — zawiera implementacje funkcji aktywacji (`ReLU`, `Sigmoid`, `Tanh`, i `Softmax`) wraz z metodami wyznaczania ich pochodnych na potrzeby propagacji wstecznej.
- **losses** — zawiera implementacje funkcji straty: błąd średniokwadratowy (`MSE`), binarna (`BCE`) i kategoryczna entropia krzyżowa (`CCE`).
- **optimizers** — implementuje algorytm *Stochastic gradient descent*.
- **mlp** — zamyka funckjonalność sieci w klasie `MLP`, przechowując listę warstw i implementując: propagację w przód, propagację wsteczną oraz kroki optymalizatora.
- **utils** — zawiera funkcje pomocnicze, w tym tworzenie mini batch'y, narzędzia do kodowania one-hot, metryki ewaluacyjne oraz ustawianie globalnego ziarna losowości.

Proces uczenia modelu sterowany jest przez metodę `MLP.fit()`, która w trakcie iteracji po kolejnych mini batchach i epokach na bieżąco zapisuje postępy uczenia. Wszystkie metadane zbierane są w formie słownika. Wśród jego kluczy znajdują się m.in. funkcje straty (w skali poszczególnych mini batchy oraz średnie na epokę dla zbioru treningowego i walidacyjnego) wprowadzone metryki (np. *Accuracy* dla klasyfikacji), normy Frobeniusa macierzy wag (dla wizualizacji postępów uczenia) oraz zmiany wartości kilku wybranych z góry wag sieci.

3 Eksperymenty — Zagadnienie regresji

3.1 Zbiory danych i metodyka badawcza

3.2 Wpływ funkcji aktywacji w warstwach ukrytych

3.3 Wpływ architektury sieci: głębokość i szerokość

3.4 Wpływ miary błędu (funkcji straty)

3.5 Wpływ rozmiaru zbioru danych (opcjonalnie)

3.6 Podsumowanie i wnioski z części regresyjnej

4 Eksperymenty — Zagadnienie klasyfikacji

4.1 Zbiory danych i metodyka badawcza

4.2 Wpływ funkcji aktywacji w warstwach ukrytych

4.3 Wpływ architektury sieci: głębokość i szerokość

4.4 Wpływ miary błędu (funkcji straty)

4.5 Podsumowanie i wnioski z części klasyfikacyjnej

5 Podsumowanie ogólne i wnioski końcowe

Literatura

[1] The Deep Hub, *He and Xavier Weight Initialization Functions*, https://medium.com/thedeephub/he-and-xavier-weight-initialization-functions-acedc5322ce5.